Öğrenme fazları: 1 • Ön-öğretim | 2 • İçerik | 3 • Katılım | 4 • Değerlendirme | 5 • İzleme

1 • Ön-öğretim **Ön Bilgi:** Hafta 3 sınıf/örtük `this`/`private` alan/kurucu ve Hafta 4 `const` üye doğruluğu & kopyalama/atama üstüne kuruludur. Operatörler de birer üye/serbest fonksiyondur — aynı kurallar (`const` doğruluğu, referans vs değer, örtük `this`) burada da geçerlidir.

» Operatör Yükleme Nedir?

Temel Fikir

Dilin yerleşik bir sembolünü (`+`, `==`, `<<`, `[]`) kendi tipiniz için `operator+` gibi özel isimli bir fonksiyonla anlamlandırırsınız. Derleyici `a + b` ifadesini üye ise `a.operator+(b)`, serbest ise `operator+(a, b)` çağrısına çevirir — sözdizimsel şeker, okunabilirliği ciddi artırır. Süreklilik: Hafta 1–2'de `Nokta c = topla(a, b);` yazıyordunuz; aynı fikri artık `Vektor2B c = a + b;` biçiminde ifade edersiniz.

» Üye mi, Serbest mi? Seçim Tablosu

Operatör	Nerede	Neden
`[] () = ->`	üye *zorunlu*	dil böyle şart koşar
`<< >>`	serbest (friend)	sol işlenen akış, siz değil
`+ - *`	üye ya da serbest	simetri için serbest (`3 * v`)
`+= -=`	üye	nesneyi yerinde değiştirir
`== <=>`	üye	`= default` ister

Üç Ayrım

Üye: sol işlenen sizin tipiniz; örtük `this` sol işlenendir. **Serbest:** sol işlenen sizin tipiniz DEĞİL (ör. `std::cout`) ise üye OLAMAZ. **friend:** serbest fonksiyon `private` alanlara erişecekse sınıf ona isim isim izin verir; yalnız `public` arayüzü kullanan serbest fonksiyon friend GEREKTİRMEZ.

» Kanonik İmzalar (Ezberleyin)

```
// deger dondurur (yeni nesne):
Vektor2B operator+(const Vektor2B&) const;
// referans dondurur (yerinde, zincir):
Vektor2B& operator+=(const Vektor2B&);
// C++20: tek tanim -> alti operator:
bool operator==(const Vektor2B&) const = default;
auto operator<=>(const Vektor2B&) const = default;
// ikili []: const + non-const cift:
int& operator[](int); // yazma
int operator[](int) const; // okuma
// akis: serbest + friend (uye OLAMAZ):
friend std::ostream& operator<<(
    std::ostream&, const Vektor2B&);
```

» Değer mi, Referans mi Döndürürsünüz?

Kural

Değer: `+`, `-`, `*` — bağımsız *yeni* nesne. **Referans** (`*this` döndürür): `+=`, `=`, non-const `[]` — var olan nesneyi verir, zincirletir. **const:** salt-okunurlar (`+`, `-`, `*`, `==`) `const`; değiştirenler (`+=`) DEĞİL.

» C++20 <=> (Spaceship)

Tek Tanımdan Altı Operatör

`auto operator<=>(const Vektor2B&) const = default;` — derleyici `< <= > >=`'i türetir. Üyeler bildirim sırasına göre *sözlüksel* (lexicographic) karşılaştırılır: önce 0. bileşen, eşitse 1. bileşen. Ayrıca `operator==`'i de `= default` yazın: yalnız eşitlik sorgusu üç yönlü hesaptan ucuzdur.

Sonuç Tipi

Tam sayı alanlar → `std::strong_ordering`; kayan noktalı alanlar (`NaN` mümkün) → `std::partial_ordering`. Amaç geometrik değil: `std::sort` / `std::set` için tutarlı bir *toplam sıralama*dır.

Dikkat

`= default`, yalnız üye sırasına göre karşılaştırma *anlamsal* sırayla örtüşüyorsa doğrudur. Bir `Kesir(pay, payda)`'da sözlüksel sıra sayısal sırayla örtüşmez ($1/2$ ile $1/3$) — orada `<=>` ELLE, çapraz çarpma ile yazılır.

» Tam Değer Tipi İskeleti

```
namespace geometri {
class Vektor2B {
public:
  explicit Vektor2B(int x = 0, int y = 0);
  Vektor2B operator+(const Vektor2B&) const;
  Vektor2B& operator+=(const Vektor2B&);
  Vektor2B operator*(int) const; // v * 3
  bool operator==(const Vektor2B&) const = default;
  auto operator<=>(const Vektor2B&) const = default;
  int& operator[](int);
  int operator[](int) const;
  friend std::ostream& operator<<(
     std::ostream&, const Vektor2B&);
private:
  std::array<int, 2> bilesenler_;
};
// serbest, friend DEGIL: uyeye devreder
Vektor2B operator*(int k, const Vektor2B& v) {
  return v * k; // 3 * v simetrisi
}
} // namespace geometri
```

» Kanonik Gövde Kalıpları

```
Vektor2B& operator+=(const Vektor2B& sag) {
  bilesenler_[0] += sag.bilesenler_[0];
  bilesenler_[1] += sag.bilesenler_[1];
  return *this; // zincir icin
}
std::ostream& operator<<(std::ostream& c,
                         const Vektor2B& v) {
  c << '(' << v.bilesenler_[0] << ", "
    << v.bilesenler_[1] << ')';
  return c; // zincir: c << v1 << v2
}
```

» Sık Yapılan Hatalar

Ortak Hata / Tuzak

- `operator+`'dan yerele *referans* döndürmek → sarkan (dangling) referans. Yeni nesne = *değer* döndürün.
- `operator<<`'i üye yazmak → sol işlenen `std::ostream`, sizin tipiniz değil; serbest olmak ZORUNDA.
- `3 * v` için yalnız üye `operator*` → derlenmez; simetri için serbest `operator*(int, const Vektor2B&)` gerekir.
- `operator+=`'da `return *this;` unutmak → zincir bozulur.
- Salt-okunuru `const` işaretlememek → `const` nesnede çağrılamaz.

» Ne Zaman Hangisi?

Hızlı Rehber

- Sizin tipiniz + değiştiriyor → **üye** (`+=`, `[]`).
- Sizin tipiniz + değiştirmiyor → üye/serbest; simetri için **serbest** (`3 * v`).
- `ostream`/`istream` → **serbest + friend**.
- Karşılaştırma → `<=>` *ve* `==`, ikisi de `= default`.

5 • İzleme **Nerede Kullanılır:** Bir *değer tipini* (`Vektor2B`, `Kesir`, `Para`) yerleşik tipler gibi doğallaştırmak istediğinizde kullanın: aritmetik/atama için `+ - * +=`; `std::sort`/`std::set` içinde sıralamak için `<=>` ve `==`; akışa yazmak için serbest+friend `<<`; konumsal erişim için `[]`. Okunabilirlik artmıyorsa (zoraki sembol) yüklemeyin; isimli üye fonksiyon yeğdir.